Angelos Michalopoulos
12/12/2025
ITC4558A1 - HIGH PERFORMANCE COMPUTING
Dr. Christou

HIGH PERFORMANCE COMPUTING
Final project

Part A:

A1: The A-Priori algorithm

For the first exercise in the final project, I chose the A-priori algorithm. The algorithm is meant to calculate the frequent item sets in a data set. This is achieved by identifying the individual items that appear most frequently in the data set and then by using larger item sets with the condition that the item sets appear often in the data set.
The algorithm works as follows:
You input a database B with a list of transactions. Then the algorithm scans the database and counts how many times each item appears in the database. It also only keeps those which are equal or larger than the minimum count threshold and puts them in a list that could be called list1.
Afterwards, for the different sizes of candidates that we want it generates the candidates and using the apriori logic, where if an itemset is frequent all its subsets are also frequent, it removes the candidates that do not exist in the list list1. Then the algorithm scans the database again, and for each transaction in the database, identifies the candidates that exist and increments the number of times they appear. Then the code is meant to remove the candidates that appear fewer times than the minimum count allows (for example if a candidate exist only once (1) and the minimum count is two (2) that candidate is removed).
Both the time and space complexity of this algorithm is $O(2^{|D|})$ which is of exponential complexity. |D| represents the width or the total number of items that are in the data set. The performance is dominated by the number of scans that occur in the Data base and the number of candidates that each level has.

A2: 3 techniques for parallelizing

The three techniques that I will discuss for the final project are, Shared memory with multithreading, distributed memory with message passing and divide and conquer with the Fork/Join model.

1. Multithreading: The way I would parallelize the Apriori algorithm with multiple threads is by splitting the database into thread chunks where each thread would be responsible for generating the candidates that appear in that particular thread. Then the thread would also have to count the number of times each candidate appears in that thread chunk and then combine the finding by joining the threads (with thread.join()).
    a. Pros in terms of speedup and efficiency:
        i. With the use of multiple threads, the same memory is shared as well as the candidate structure, meaning that it can be done locally.
        ii. There is also no network communication but only in-memory synchronization which makes the program run a lot faster thus affecting the speedup.

b. Cons in terms of speedup and efficiency:
    i. The first con is that with this technique there is limited scalability since it is bound by the number of cores and memory that exist on a single machine.
    ii. Updates might cause synchronization overhead if they have to happen frequently.
    iii. This doesn't solve the complexity of the original algorithm since it still has to do multiple brute force scans of the database.
c. Implementation of the frameworks (Spark and MPICH)
    i. Spark:
    Spark doesn't manually create threads and uses the thread pool that exists inside each executor. Multithreading can be done by running the task using the local[N] mode to use the number of cores you want to use and then each partition is processed by one thread. The pros with spark are that you can express the code as RDD transformations such as .cartesian(). The cons with spark are that the overhead might cause the code to slow down if doesn't have a big enough database and has less control over thread placement.
    ii. MPICH:
    MPICH framework is process-centric since it is a MPI implementation which means that it isn't thread-centric. However, you can do a hybrid version where each MPI uses threads and the java thread Internally. The pros of MPICH are potentially useful if each node runs multiple threads over a local subset of data but the cons are that it makes the process more complex.

2. Message Passing: This is a classic distributed memory pattern which will part the transactions set across a lot of message passing processes and each prosses will compute the local counts and the current candidate. Then a global reduction to combine the local support counts will happen and this is repeated for all the sizes of itemsets that we want. I would use a message passing coordinator that has a send data function and a receive data function so that the workers can communicate between them and use the data they require.
    a. Pros in terms of speedup and efficiency:
        i. The first pro is scalability because it can scale across multiple nodes, and each node only hold part of the transaction in the database.
        ii. It helps with large datasets which don't fit in a single machine.
    b. Cons in terms of speedup and efficiency:
        i. There is communication overhead because it has to distribute candidate sets to all the processes.
        ii. One again it doesn't fix the main problem of the serial algorithm, which is the complexity of it, since it still has to scan the data multiple times making it slow.
    c. Implementation of the frameworks (Spark and MPICH)

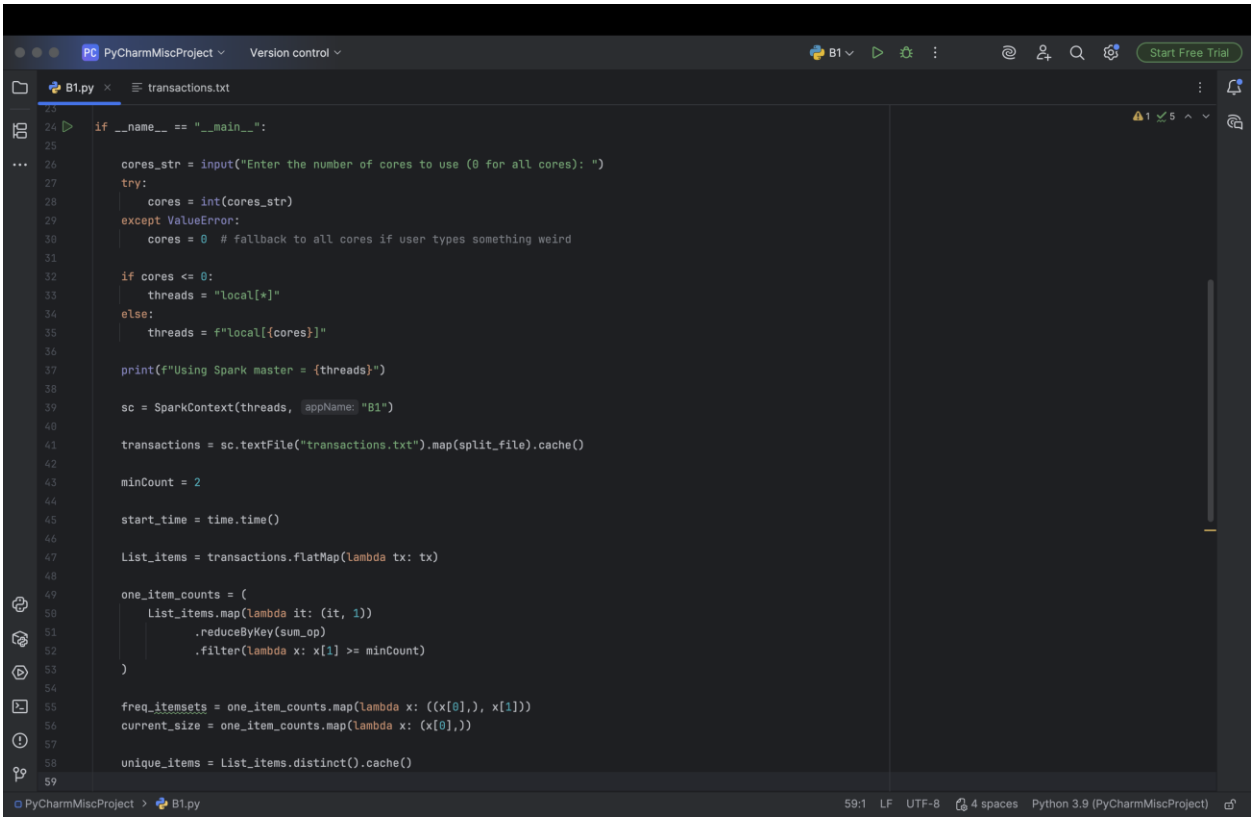
i. Spark:
   In spark the message passing happens implicitly through some commands such as groupByKey or join and through the broadcasting variables which is mostly done for the candidate sets. The pros are that there are no explicit send and receive data functions since spark does that on its own and RDD allows for fault tolerance. However, one con is that you have less control over low-level communications.
ii. MPICH:
   The MPICH framework is the natural framework for message passing because it performs one process per core. That means that the pros are that we have control over the communication of the code and data but also high performance and a low overhead.

3. Divide and conquer (Fork and join model): With the fork and join model we have structured parallelism which recursively splits (fork) the work into sub-tasks and each task processes a subset of the task. When it is complete it is combined (join). For the apriori algorithm a possible strategy is as follows. A recursive task that holds a range splits the transactions and then a second recursive task splits the responsibilities for the generation of the candidates and the number of times each candidate exists and then they all join for the final result.
   a. Pros in terms of speedup and efficiency:
      i. Because fork/join automatically balances the load of each task so that no task has more work than the other it fixes the speedup since each task can start and complete their work together, therefore no task has to wait.
      ii. Because it has a clear divide and conquer pattern it is more structured for parallel programing.
   b. Cons in terms of speedup and efficiency
      i. There is still overhead because of the task creation and joining which has a cost making it less efficient. Also, since it still has to scan the database for multiple sizes, each time you have to overcome the overhead.
      ii. It is limited to a single machine.
   c. Implementations for the frameworks:
      Neither Spark nor MPICH can do fork join since it is a model on its own.

Part B1:

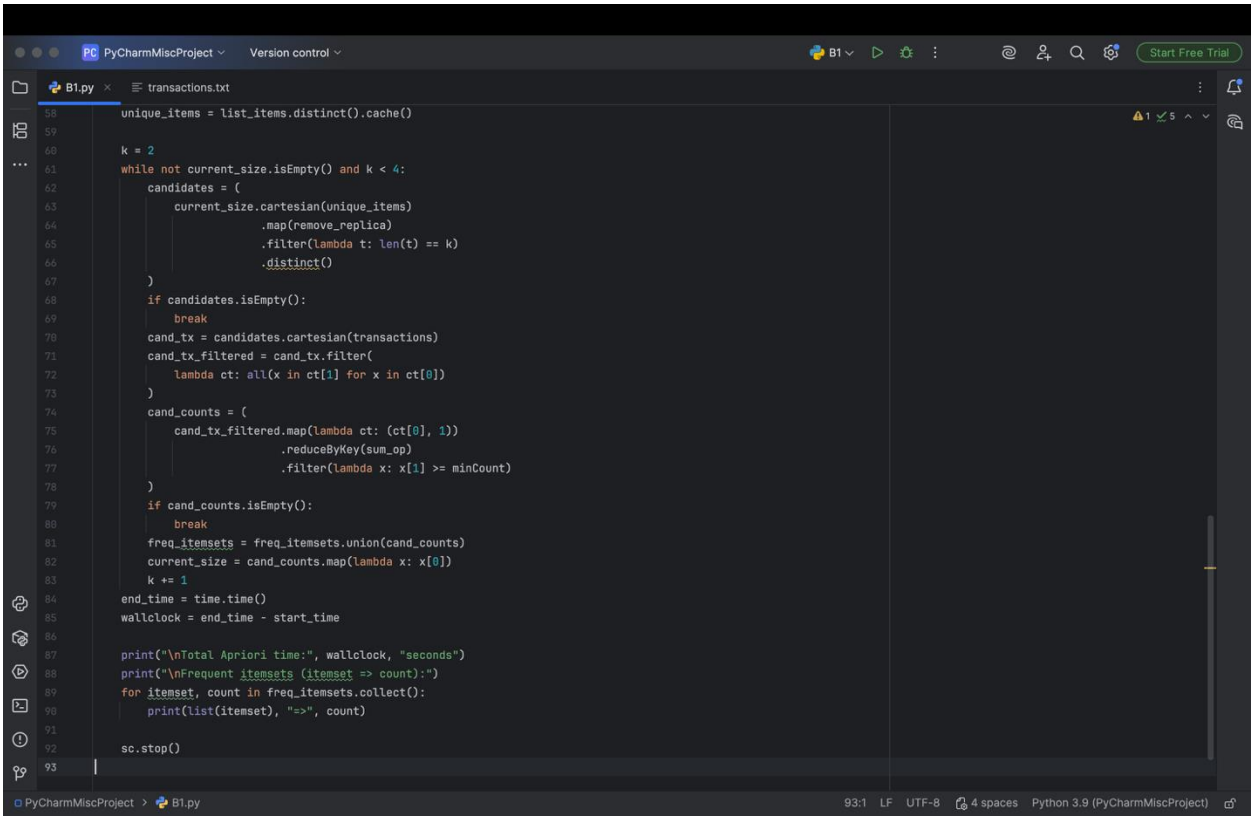
B1a: the code

The framework I chose to use for the B1 assignment is the Apache spark code which I can write in python which I am more proficient in that C++.

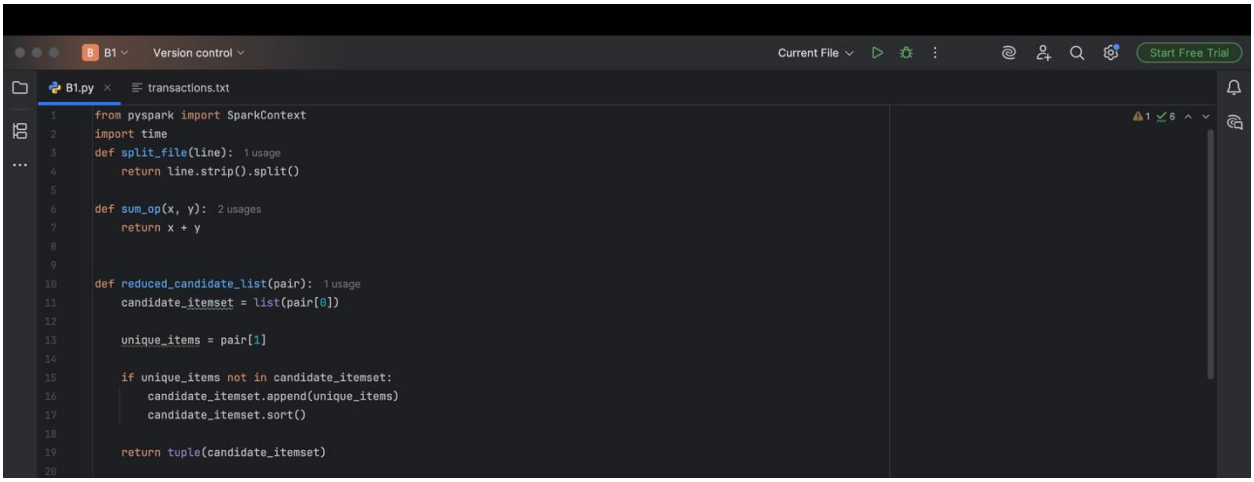


The screenshot above shows the main function of the pyspark implementation for the Apriori algorithm. It starts with asking the user to input the number of cores that they want to use for the execution of the code (with 0 being all of the cores that the machine can use, which in my case is 10). After the user chooses the number of cores the alorithm stores it in a threads = “local[{cores}]” code which defines the number of cores that the code will use in the format that spark can use. Then using SparkContext(threads, B1) it starts the spark application B1, using the number of threads that the user has requested. The next thing the code does is call the database (which is the transaction.txt file) and stores it in a variable called transaction using the function sc.textFile(“transaction.txt”).map(split_file).cache(). This function stores each line in the variable transactions using the map transformation from pyspark and the split_file helper function which I will explain latter in the report. Afterwards the code initializes a integer variable which is called minCount to 2 which is telling the code that the minimum number of times a itemset has to appear for it to count in the final output is 2. Then we start the timer for the execution of the code so we can check its execution time. The following code is how the program calculates the

number of times every single item in the transaction database appears. The flatMap transformation takes the values from the transaction variable which look like this ([A,B],[B],[A,C]) and separates each item so that then the one_item_count variable can map it so that it shows each item and the number one (1) next to it. Then by using the reduceByKey transformation function we can use the sum_op helper function and count how many times each variable appears. In my example above it would be something like this (“A,2”, “B,2”, “C,1”). Then the last spark transformation, that has to deal with counting the number of times a single item appears in the data base, is the filter transformation which filters out the items that appear less than the MinCount allows. The code that follows, has to do with preparing the data for the loop and the different size of subsets that it will have to find and count. The first function is freq_itemsets = one_item_counts.map(lambda x: ((x[0],), x[1])) which takes the results from the process for counting the number of items each time it appears and transforms them so that each result is stored as a tuple which is the standard itemset data type. Then the code does current_size = one_item_supports.map(lambda x: (x[0],)) which is responsible for extracting only the item names so that it can format them as single tuples and use it as input for generating the candidates for the following sizes (which in my code is only up to k = 3). Lastly, the unique_items = List_items.distinct().cache() function takes the list of items the code found previously and using the .disticnt() function only picks the unique ones. Using this list and the current_level variable we can calculate the number of times subsets exist in the database. The cache() function is responsible for saving the data in the memory of the machine and this is done because the list is static and will be reused constantly.



Firstly, I initialize the number of levels (or size of the sets) that the loop will be doing. The loop happens for as long as the current_size list is not empty and as long as K is smaller than 4. The code first generates the candidates by using the transformation function of current_level.cartesian(unique_items) which forms a massive product by crossing the elements of current_size with the elements of the two together and joins them. It then maps it to a single item (using the reduced_candidate_list helper function) which also removes duplicates such as ([A],[A]) by not allowing them to be created and then the .duplicates() function removes any duplicate itemsets from being present in the candidates list. Then it takes those items and only keeps the ones whose size is k (depending on the iteration k will be different). This way it creates the candidate list which then using the .cartesian() function again it crosses the candidates with the original transaction list and then filters the list by making sure that everything from candidate itemset appear and is a subset from the transactions itemset. For example if one value of the list is ([A,B],[A,B,C]) it would pick this as a candidate is a subset of the transaction. ([A,B],[A,C]) isn't since B does not exist in both itemsets. After finding the candidates that exist, the code has to count the number of times they appear and remove the ones that do not meet the criteria, and this is done the same way as above. Finally, the code adds the new results to the frequent itemset list and then like before it extracts the itemsets in the current_size variable in order to use them for generating the candidates for the next size which changes because the k is incremented by one, and the loop can start again. The last part of the code has to do with printing the results.
The last part of this code I will explain are the helper functions which appear in the screenshot bellow:

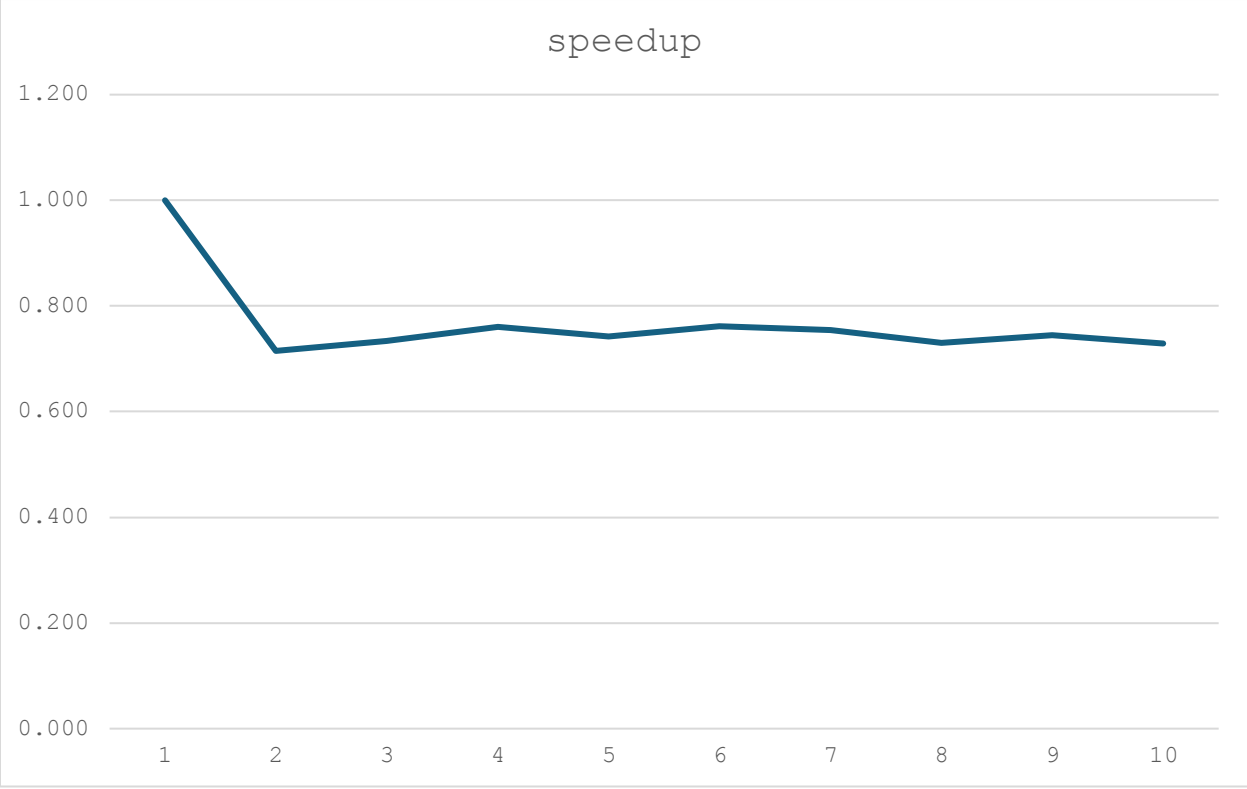


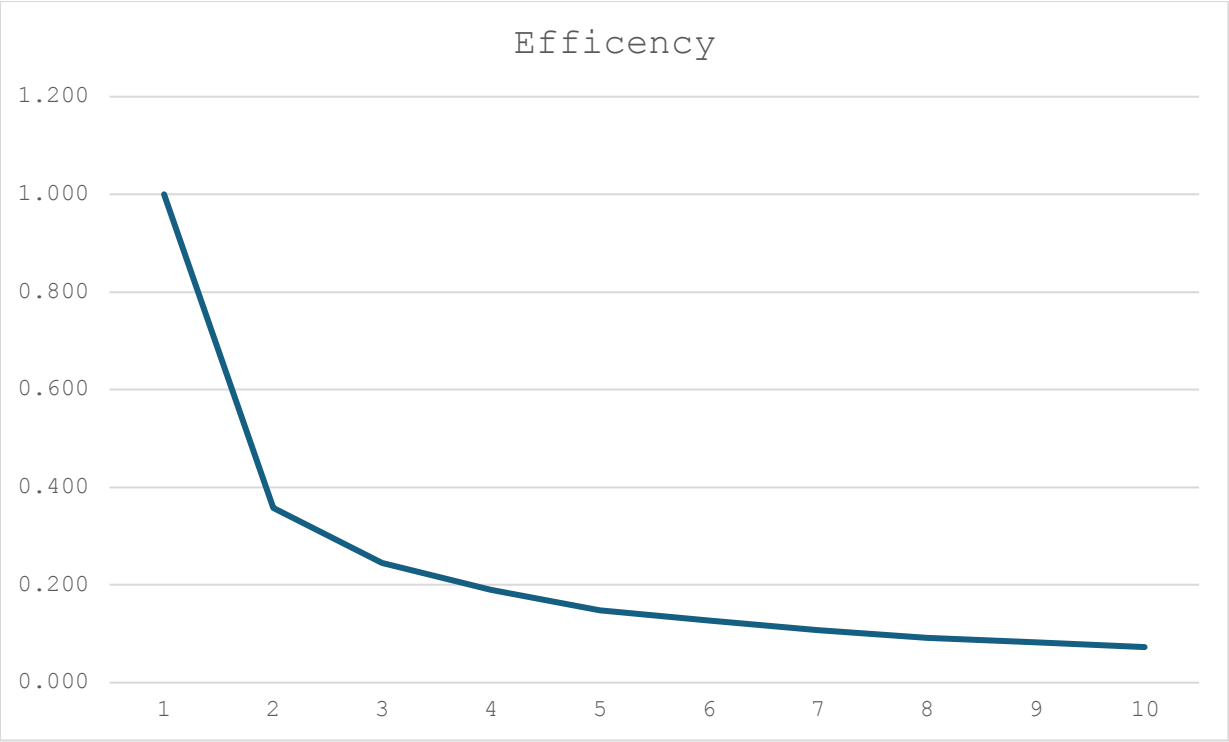
The functions are split_file, sum_op and reduced_candidate_list. Firstly, the split_file function gets a line from the database and splits the elements one by one and the sum_op aggregates the counts of all the times that an itemset apperas (or item if it is done in the moment where only one itemset is searched for) and increments the counts for example ("A,1", "A,1") becomes ("A,2"). This is done because the function is parsed though reduceByKey(sum_op) so it searches for all the itemsets that have the key "A" and increments the count by getting the first two values of the first two "A" keys and adds them. It does that for every key in the list of itemsets. Finally, the most important helper function is the reduced_candidate_list function. Its main purpose is after using the cartesian transformation to determine that the itemsets are unique and that they

are sorted so that when we use the distinct transformation later it only keeps the unique sets. It first gets the pair (which is the output from the cartesian product) and separates the pair into a list from the candidates (using the pair[0]) called candidate_itemset and then a unique item from the unique_items list (pair[1]) .It then checks it the unique item is in the Candidate list using an if statement. If the item isn't in the list it is appended to the candidate_itemset list and sorts it so that when we call the .disticnt() transformation if two item sets are the same they are removed and then it returns it as a tuple.

B1b: The speedup and the efficiency evaluations

threads	speed	speedup	efficency
1	0.66264606	1.000	1.000
2	0.92681217	0.715	0.357
3	0.90261483	0.734	0.245
4	0.87079287	0.761	0.190
5	0.89279985	0.742	0.148
6	0.87041879	0.761	0.127
7	0.87771297	0.755	0.108
8	0.907166	0.730	0.091
9	0.88972092	0.745	0.083
10	0.90935516	0.729	0.073

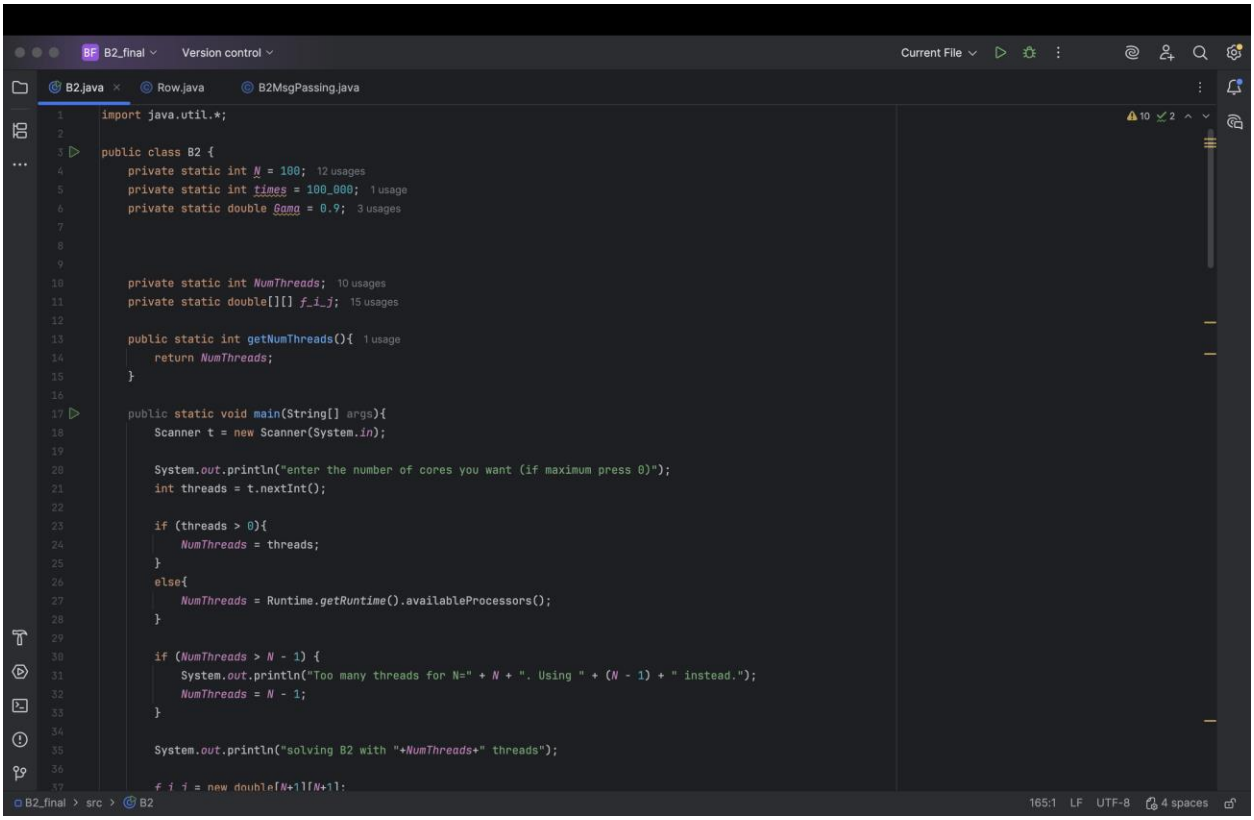




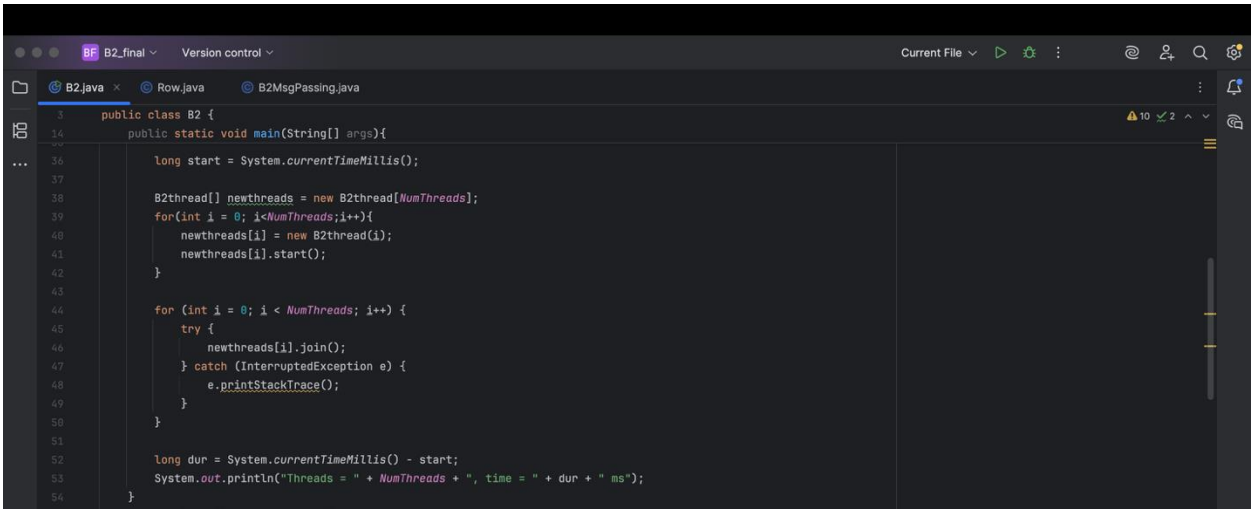
From the speedup and the efficiency, we see that the code struggles with when more than one core is being use. This is because of the spark overhead and the small dataset that I am using to test the code. Apache spark was meant to be used for very big datasets and since my dataset only has 8 transactions it means that the overhead for initializing the SparkContext components and scheduling tasks cannot be overcome. This occurs from the fact that the dataset is very small and that negatively impacts the speedup which also impacts the efficiency. Also adding the algorithm's innate space and time complexity adds to this fact when trying to parallelize it.

Part B2:

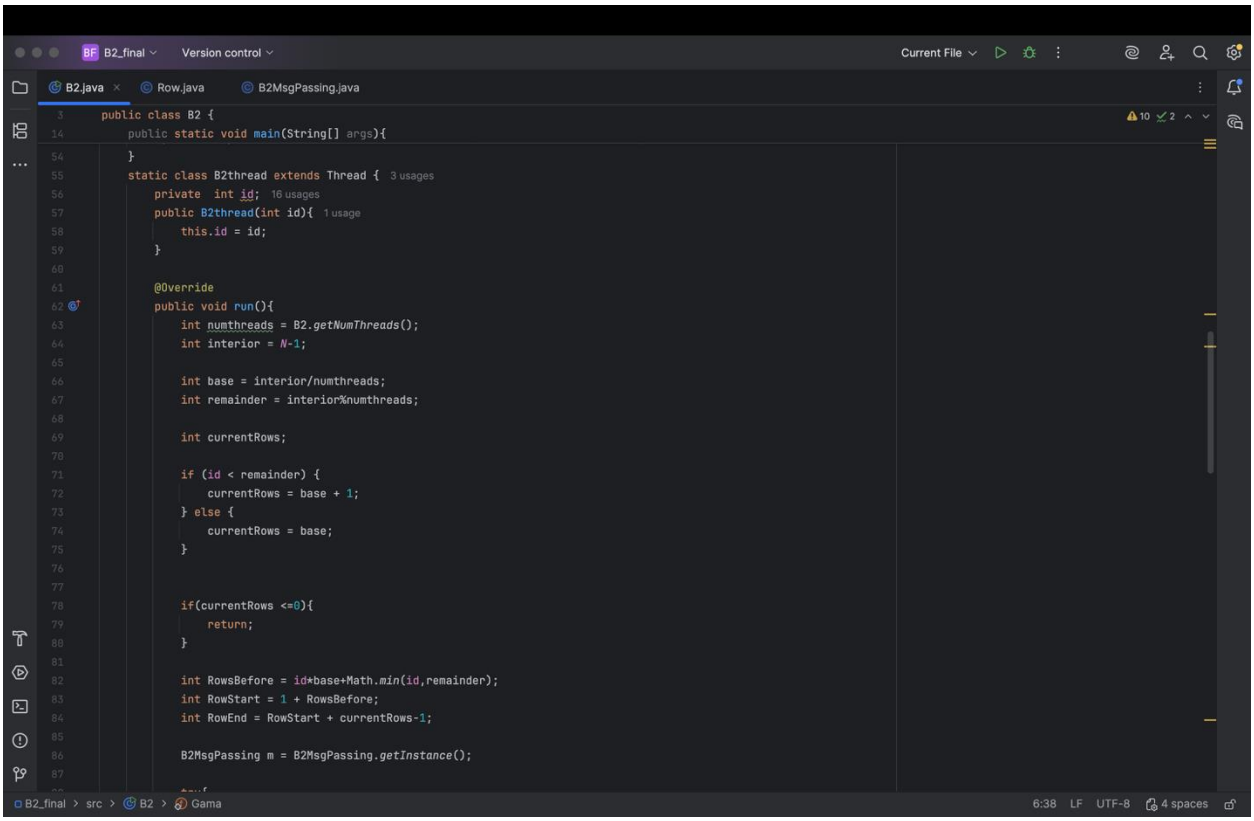
B2a: the code



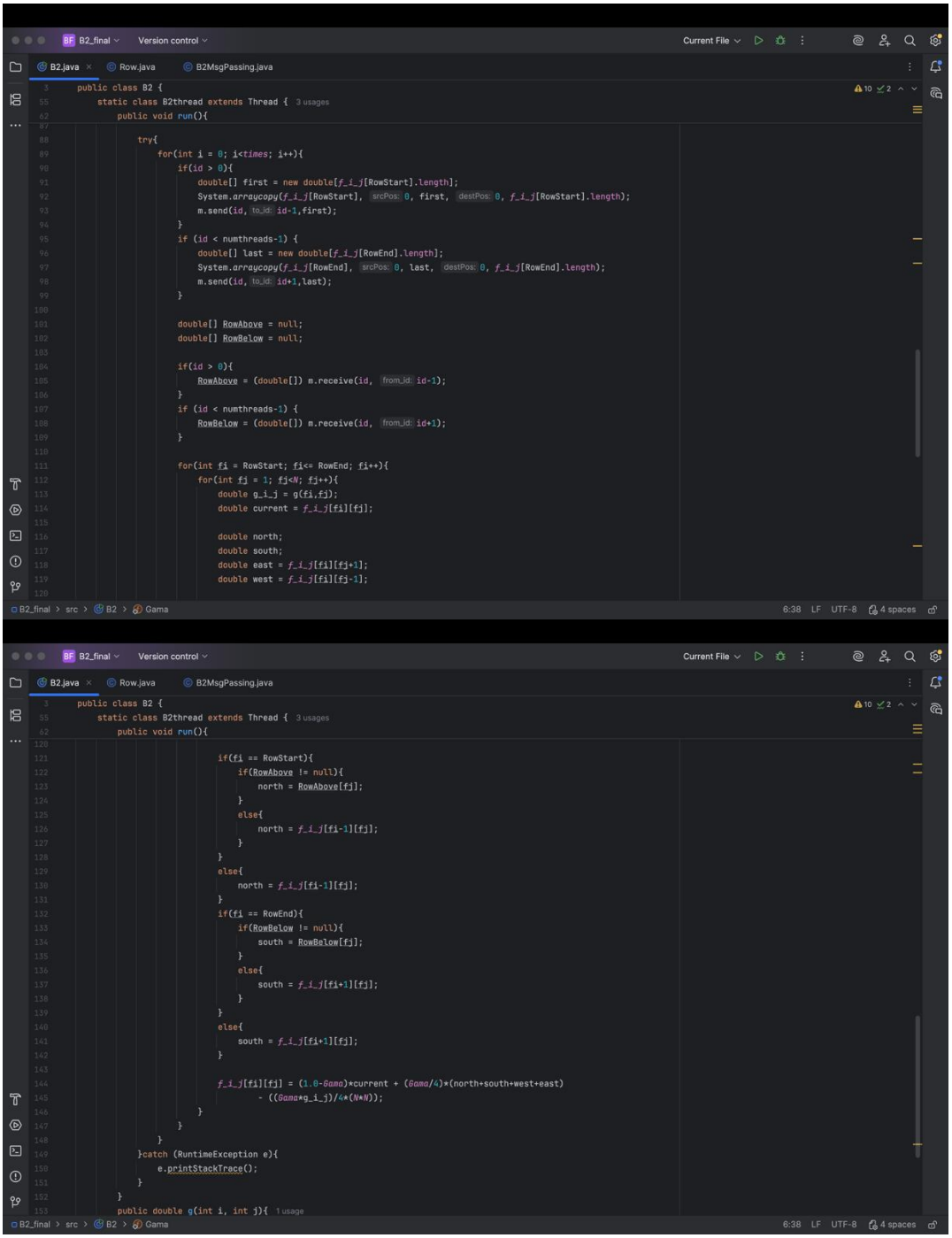
The screenshot above shows the first thing that the code does which is initialize some values such as N (the size of the grid), times (the number of iterations that the code has to do) and Gama (which is a given number that is used in the function for the update of the grid). Two more values that are created are the gird f_i_j and the numThreads variable which are not given a value now but later through the execution of the code. I also create a getNumThreads function which I will also use later (for when I create the B2Threads and run them). In the main class the code asks the user to input the number of cores (or threads) that they want to use for the execution of the code, where the user can either enter a number bigger than or equal to 1 for that specific number of threads or the number 0 which gives you the default maximum number of threads that your machine has (in this case as mentioned above it is 10). This is done using the Runtime.getRuntime().availableProcessors(); code. I then had a quick if statement control where if the user puts more threads than the number of rows in the grid then it stops and uses the maximum number of threads that it can have which is N-1. This is done because if there were more threads than rows the code wouldn't be able to split the work to all the threads correctly. The next thing the code does is initializing the grid with N+1 just as the project description requested (f_i_j = new double[N+1][N+1] and java initializes every double with 0.0 (unless it is initialized with a number).



The next phase of the code is the creation and starting of the threads that it is going to use to complete the task. Firstly, I created a long variable called start with the value System.currentTimeMillis() which starts the timer to keep track of the execution time and can calculate the speedup and efficiency of the code. The next part is the creation of the threads where I create an B2Thread[] object called newthreads that has NumThreads (whatever number the user chose before) as the number of threads created. In the for loop following, I create every individual thread (newthread[i] = new B2Thread(i)) with the thread number designated as its id. I then use the thread function .start() to start the run function inside the B2Thread. The next for loop is responsible for joining the threads together and ending the execution of the b2Thread run function and this is done with the following code (newthreads[i].join();). Afterwards I create a second long variable called dur where I get the total execution time of the code after the start (System.currentTimeMillis() – start) in milliseconds and I print the final output of the code where I show how long it took the code to complete the task for that specific number of threads.



The following code that is in the B2 class (the main class of the project) has to do with the B2Thread and the actual task of the code. Firstly, the B2Thread class has as a signature the id that corresponds to each thread (0 for the first thread, 1 for the second… N-1 for the Nth thread) and a constructor to initialize the id in the B2Thread class. The following function is the public void run() function which is the function that contains the task and is executed when the program calls the .start() function above. It starts with getting the number of threads that the user has decided they want to use and designates the interior cells of the grid (in the case of the project it is 99) as we only want to update the cells of the grid for when I != 0 or N and when j != 0 or N (meaning the interior rows) and keeps the outer rows and columns 0.0. The code then designates the base number of rows that each thread has to deal with (for example if numThreads is 10 and N-1 = 99, each thread has to be responsible for 10 threads as a base) and then the remainder variable sees how many rows would remain (for example keeping with threads 10 and N-1 = 99 the remainder would be 9 remaining rows that the code has to place in the already existing rows). Then I create a variable called currentRows to show the rows that each thread is currently responsible for. And with the remainder I deal with them with the if statement when I spread the remaining rows in the threads one by one. For example, if there are 9 rows remaining the first 9 rows all get one more row they have to deal with, otherwise they just get the base rows. I then calculate the rows that where before for each thread using the int RowsBefore = id*base+Math.min(id,remainder); code so that I can correctly calculate at which row each thread has to start at (which is stored in the RowStart variable) and end with (which is stored in the RowEnd variable). Finally, for this screenshot I create a message passing object using the B2MgPassing.getInstance() function.



```
public class B2 {
    static class B2thread extends Thread {  3 usages
        public void run(){

            try{
                for(int i = 0; i<times; i++){
                    if(id > 0){
                        double[] first = new double[f_i_j[RowStart].length];
                        System.arraycopy(f_i_j[RowStart],  srcPos: 0, first,  destPos: 0, f_i_j[RowStart].length);
                        m.send(id,  to_id: id-1,first);
                    }
                    if (id < numthreads-1) {
                        double[] last = new double[f_i_j[RowEnd].length];
                        System.arraycopy(f_i_j[RowEnd],  srcPos: 0, last,  destPos: 0, f_i_j[RowEnd].length);
                        m.send(id,  to_id: id+1,last);
                    }

                    double[] RowAbove = null;
                    double[] RowBelow = null;

                    if(id > 0){
                        RowAbove = (double[]) m.receive(id,  from_id: id-1);
                    }
                    if (id < numthreads-1) {
                        RowBelow = (double[]) m.receive(id,  from_id: id+1);
                    }

                    for(int fi = RowStart; fi<= RowEnd; fi++){
                        for(int fj = 1; fj<N; fj++){
                            double g_i_j = g(fi,fj);
                            double current = f_i_j[fi][fj];

                            double north;
                            double south;
                            double east = f_i_j[fi][fj+1];
                            double west = f_i_j[fi][fj-1];
```



```
public class B2 {
    static class B2thread extends Thread {  3 usages
        public void run(){

                            if(fi == RowStart){
                                if(RowAbove != null){
                                    north = RowAbove[fj];
                                }
                                else{
                                    north = f_i_j[fi-1][fj];
                                }
                            }
                            else{
                                north = f_i_j[fi-1][fj];
                            }
                            if(fi == RowEnd){
                                if(RowBelow != null){
                                    south = RowBelow[fj];
                                }
                                else{
                                    south = f_i_j[fi+1][fj];
                                }
                            }
                            else{
                                south = f_i_j[fi+1][fj];
                            }

                            f_i_j[fi][fj] = (1.0-Gama)*current + (Gama/4)*(north+south+west+east)
                                    - ((Gama*g_i_j)/4*(N*N));
                        }
                    }
                }
            }catch (RuntimeException e){
                e.printStackTrace();
            }
        }
        public double g(int i, int j){  1 usage
```

The two screenshots above correspond to the code that is responsible for calculating and updating the grid. The entire code is in a for loop that is running for 100.000 iterations just as the project demands. Following, there are two if statements that check whether the thread is the first or the last thread that is created. If the id of the thread is over 0 it copies the threads first row and using the send function from the MsgPassing code it sends it to the previous id. This occurs in order for that id to update the last row it is responsible for, since it has to use the south

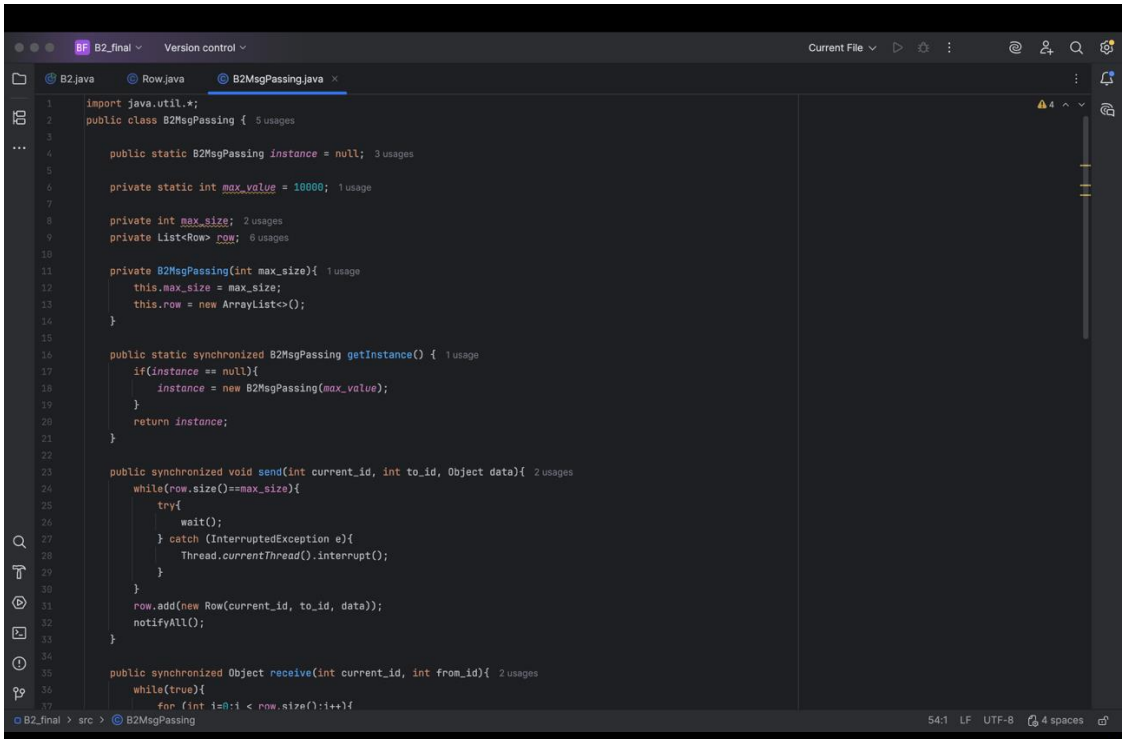
neighbors of the previous iteration. Then it checks if the thread is the last one. If it isn't the last, it does the same process with the previous if statement but sends its last row to the next thread as it needs the north neighbors of its first row in order for it to do the grid updates. Afterwards, I create two double variables (RowAbove and RowBelow) that are initially initialized as null to show that they don't have any values as of now. Using a similar if statement as above which sees if the thread is either the first one of the last one, the code gets the data that was sent previously and places it to the correct variable. If the id is not the first one it means that it takes a row from the previous thread so a row from the above thread is stored in the RowAbove variable using the m.recive(id,id-1) message passing function. If the thread is not the last one, it needs a thread from the next row so a row from the next thread is stored in the RowBelow using this m.recive(id,id+1) function. Following this, is the for loop which is done for the interior of the grid (so 99 rows and 99 columns) which is responsible for the calculation and the updates of the grid. It also finds the current row with current = f_i_j[fi][fj] and Initializes g_i_j which is a function that is needed to update the grid. This is done using this function:

$$f_{\{i,j\}}(t+1) = (1-\gamma)f_{\{i,j\}}(t) + \frac{\gamma}{4}\left[f_{\{i+1,j\}}(t) + f_{\{i-1,j\}}(t) + f_{\{i,j+1\}}(t) + f_{\{i,j-1\}}(t)\right] - \frac{\gamma g_{\{i,j\}}}{4N^2}$$

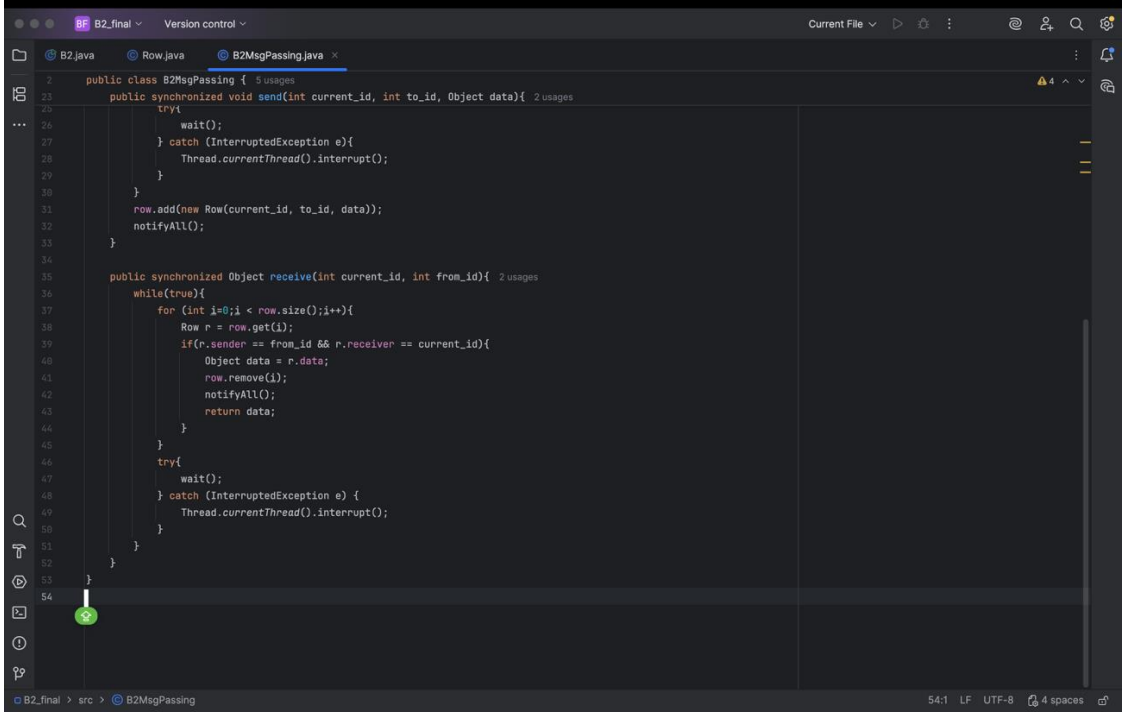
Where $f_{\{i+1,j\}}(t)$ is the north position of the current iteration of the grid, $f_{\{i-1,j\}}(t)$ is the south position, $f_{\{i,j+1\}}(t)$ is the east and lastly $f_{\{i,j-1\}}(t)$ is the west. In the code for east and west we just store the values of the grid in two double variables (east and west) just as follows double east = f_i_j[fi][fj+1]; and the same is done for the west variable. When it comes to the north and south it first has to see if fi is the first row of the thread. If this is the case and its RowAbove is not null (meaning it is not the very first row of the first thread) it makes the north position the data from RowAbove. If it is the first row from the first thread (RowAbove is null), then it does the same as with the east and west (north = f_i_j[fi-1][fj];). The same is also performed if it isn't the first row of the thread. A similar process is done for the south position for each thread, but it sees if the fi is the last row and uses the RowBelow variable. After it has given the values to all the positions, the code executes the function for updating the grid which is (f_i_j[fi][fj] = (1.0-Gama)*current + (Gama/4)*(north + south + west + east)- ((Gama*g_i_j)/4*(N*N));), where g_i_j is the function mentioned above and is shown in the following screenshot.

```
}
public double g(int i, int j){  1 usage
    double x = (double) i / N;
    double y = (double) j / N;
    return -2.0 * (x * (1.0 - x) + y * (1.0 - y));
}
```

The next code that I have to explain is my B2MsgPassing code which is meant to be the message passing coordinator for the project and make sure that the threads can communicate between them. This part of the code was based it on this code I found online (https://www.geeksforgeeks.org/java/message-passing-in-java/) but changed it in order to fit my needs.



```java
import java.util.*;
public class B2MsgPassing {

    public static B2MsgPassing instance = null;

    private static int max_value = 10000;

    private int max_size;
    private List<Row> row;

    private B2MsgPassing(int max_size){
        this.max_size = max_size;
        this.row = new ArrayList<>();
    }

    public static synchronized B2MsgPassing getInstance() {
        if(instance == null){
            instance = new B2MsgPassing(max_value);
        }
        return instance;
    }

    public synchronized void send(int current_id, int to_id, Object data){
        while(row.size()==max_size){
            try{
                wait();
            } catch (InterruptedException e){
                Thread.currentThread().interrupt();
            }
        }
        row.add(new Row(current_id, to_id, data));
        notifyAll();
    }

    public synchronized Object receive(int current_id, int from_id){
        while(true){
            for (int i=0;i < row.size();i++){
```



```java
public class B2MsgPassing {
    public synchronized void send(int current_id, int to_id, Object data){
            try{
                wait();
            } catch (InterruptedException e){
                Thread.currentThread().interrupt();
            }
        }
        row.add(new Row(current_id, to_id, data));
        notifyAll();
    }

    public synchronized Object receive(int current_id, int from_id){
        while(true){
            for (int i=0;i < row.size();i++){
                Row r = row.get(i);
                if(r.sender == from_id && r.receiver == current_id){
                    Object data = r.data;
                    row.remove(i);
                    notifyAll();
                    return data;
                }
            }
            try{
                wait();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }
}
```

The code for this class starts by initializing the instance. The class uses the Singleton design pattern to ensure only one instance of the message passing object exists. The instance variable is initialized as null. It also initializes the Max value of the size that it can receive in the queue. It also creates a list object with a class row that shows the data it has to send, which is the ID of the thread, the send to id of the target thread, and the data of the row that it has to send. It then has a constructor to set it all up. The first function in this class is the get instance function which basically creates the object by creating a new message passing object with that instance. After it gets the instance, the next function is the send function which is meant to send the data from one thread to the other. In the signature of the send function there is the current id, the target id, and the data it has to send. The function has an if statement that checks If the queue has already reached its Max size. If it has then it waits until it gets notified that you can use the function where it then stores the array of the current ID, the To_ID and the data In the list the code created earlier. The next function is the receive function. In its signature it has the current ID and the ID it will receive data from. In it there is a for loop that compares the contents of the list with each row with the variables in the functions signature. When the ID matches the send to ID and the from ID matches the current ID in the row, it takes the data and returns it and notifies all that the receive function is free to use.

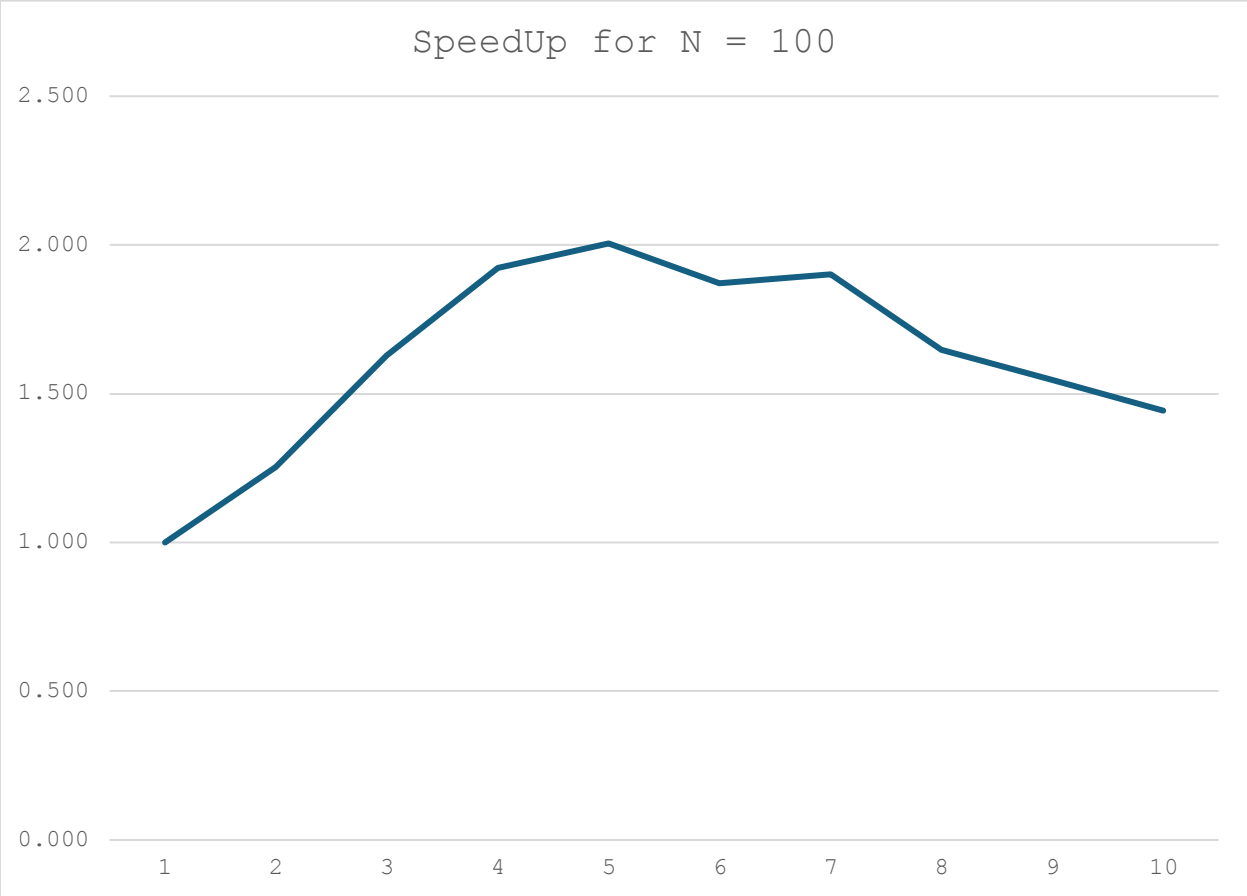
B2b: The speedup and the efficiency evaluations

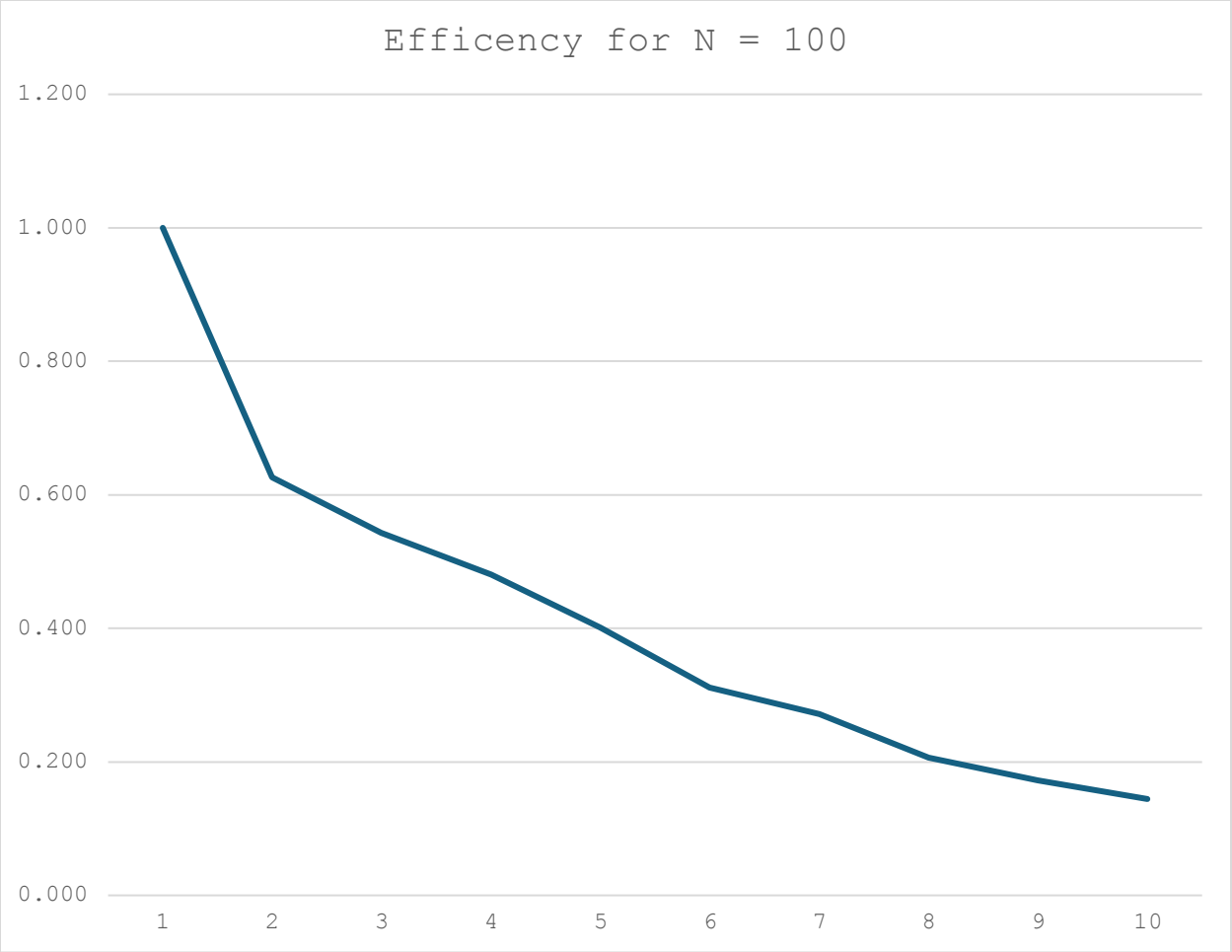
threads	speed	speedup	efficiency
1	3056	1.000	1.000
2	2439	1.253	0.626
3	1876	1.629	0.543
4	1590	1.922	0.481
5	1524	2.005	0.401
6	1634	1.870	0.312
7	1607	1.902	0.272
8	1855	1.647	0.206
9	1976	1.547	0.172
10	2117	1.444	0.144

The table above depicts the efficiency and the speedup of the code when tested with different number of cores starting with 1 core (serial execution) until the number of cores reaches the maximum number the machine has (10). As we see from the table every number of threads more than 1 is faster meaning that there is positive speedup for every percentage of parallelization of the code. It gradually increases to twice the speedup util 5 cores (which are half the cores of the machine) but following this we see a small gradual decrease in the speedup of the code every time we add a core to the execution until it reaches the maximum cores. That being said the execution time of the code never goes slower than the serial version of the code, meaning that even if the code with 10 threads is slower that that with 5 threads it is still faster than the serial

version. From the standpoint of the efficiency, we see that there is a gradual drop in efficiency even if the speed up increases until it reaches 10 threads. Even though it has a positive speed up it only uses 14% of its cores efficiently. This is to be expected since I am suing Amdals law for calculating efficiency which caps the returns as you add processors because parallelizing a task is limited by its serial portion, and since the speedup is positive put not very large to overcome this limit in efficiency.

The graph below shows the visual representation of the table for speedup and efficiency when the N of the Grid is 100 rows and 100 columns.





Sources:

1. “Message Passing in Java.” *GeeksforGeeks*, GeeksforGeeks, 11 July 2025,www.geeksforgeeks.org/java/message-passing-in-java/.
2. “Apriori Algorithm.” *GeeksforGeeks*, 21 Nov. 2025, www.geeksforgeeks.org/machinelearning/apriori-algorithm/.
3. “Apriori Algorithm.” *Wikipedia*, Wikimedia Foundation, 24 Oct. 2025,en.wikipedia.org/wiki/Apriori_algorithm.
4. “The Good and the Bad of Apache Spark.” *AltexSoft*, AltexSoft, 18 July 2023,www.altexsoft.com/blog/apache-spark-pros-cons/.
5. *2: Advantages and Disadvantages of Using MPI.* , www.researchgate.net/figure/2-Advantages-and-disadvantages-of-using-MPI_tbl4_358807869. Accessed 12 Dec. 2025.